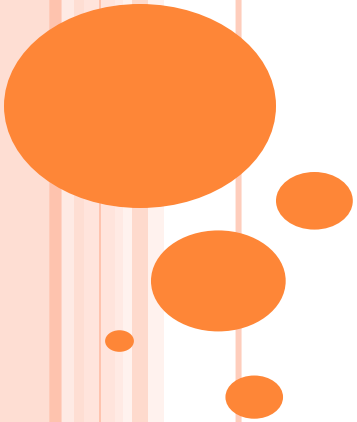
Sintaxis python



Tipos de datos y variables

¿Qué son los datos?

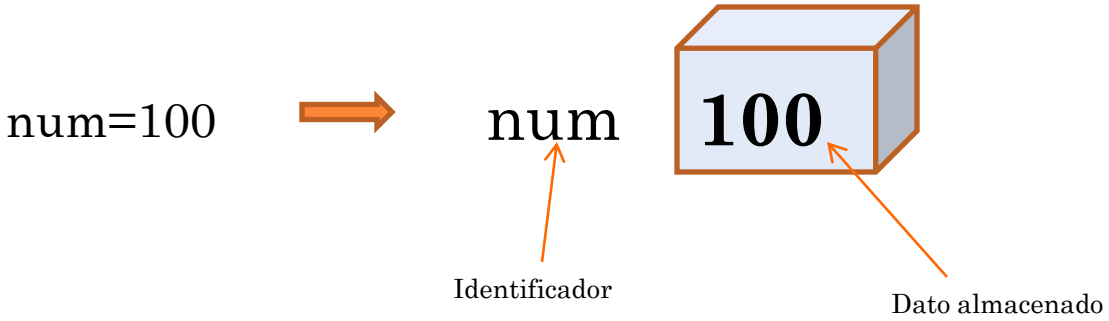
- **Información manejada dentro de un programa (números, textos, etc.)**

- **Python soporta los siguientes tipos de datos básicos:**

Enteros	Números de tipo entero, por ejemplo 10, 35, -15
Coma flotante	Números que pueden contener cifras decimales: 7.2, 5.9
Lógicos	Representan dos situaciones opuestas: True, False
Texto	Combinación de caracteres: “Esto es un texto”, “otra frase”

Las variables

- Cajas" que almacenan los datos para poder manejarlos en el programa.
- Se les asigna un identificador o nombre.
- El valor o dato se asigna a la variable mediante el signo =, de derecha a izquierda:



Identificadores permitidos

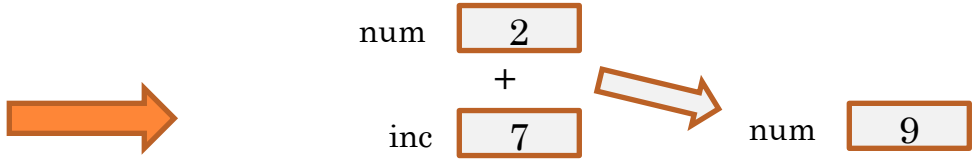
- **Se pueden utilizar cualquier combinación de caracteres alfanuméricos para definir un identificador, con restricciones:**
  - **No pueden comenzar por número**
  - **No se pueden incluir signos de puntuación**
  - **No puede coincidir con palabras reservadas**

Ejemplos válidos	Ejemplos no válidos
_n20=10 a4f=500	1r=3 m.t=10 for=4

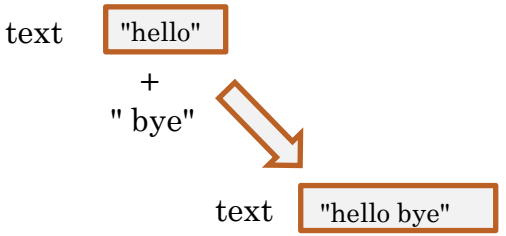
Operaciones con variables

- **Los datos se almacenan en variables para que el programa pueda realizar operaciones con ellos:**

```
num=2;inc=7
#acumula un valor en la variable
num=num+inc
```

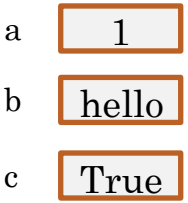



```
text="hello"
#une un texto con otro
text=text+" bye"
```



- **Python admite asignación múltiple:**

```
a,b,c=1,"hello",True
```



Entrada/salida en python

➢Python dispone de dos instrucciones para escribir datos en pantalla y leer por teclado:

- **print(dato). Imprime en pantalla el dato que se indica entre paréntesis, puede ser una variable:**

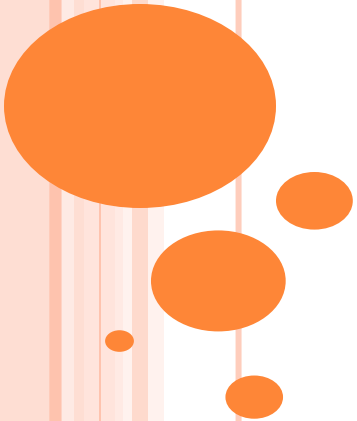
```
num=2
num=num*num #calcula el cuadrado del número
print(num) #muestra 4
print("El numero es ",num) #muestra un mensaje y la variable
print(f"El numero es {num}")  #impresión con formato
```

- **input(texto). Muestra un mensaje de texto y devuelve el dato escrito por teclado, que se deberá guardar en una variable**

```
#solicita la introducción del nombre y lo muestra
nombre=input("Dame tu nombre: ")
print("Te llamas ", nombre)
n=input("Número 1: ")  #introduce el 3
x=input("Número 2: ") #introduce el 4
print("la suma es ",(n+x)) #ojo, muestra 34. Concatena
```



Operadores



¿Qué es un operador?

- Símbolo utilizado para operar con datos dentro de un programa
- Pueden ser utilizados sobre variables
- Se distinguen los siguientes tipos:
  - Aritméticos
  - Asignación
  - Relacionales
  - Lógicos



Operadores aritméticos

- **Realizan operaciones con datos de tipo numérico.**

Operador	Significado	Ejemplo
+	Suma de números	sum=x+y
-	Resta de números	res=x-y
*	Multiplicación de números	multi=x*y
/	División de números	div=x/y
//	División con redondeo inferior	print(5//2) #2
%	Resto de la división	print (5%2) #1
**	Potencia de dos números	print(2**3) #8

Operadores asignación

- **Realizan operaciones de asignación.**

Operador	Significado	Ejemplo
=	Asigna valor de la derecha a la izquierda	data=100 a=b=c=3 #asignación múltiple
+=	Suma el dato con la variable y lo vuelca en la misma	x+=5 #equivale a x=x+5
-=	Resta el dato a la variable y lo asigna	x-=2 #equivale a x=x-2
*=	Multiplica y asigna	mul*=7
/=	Divide y asigna	div/=2
%=	Calcula resto y asigna	rest%=2
//=	División con redondeo y asigna	red//=3
**=	Potencia y asigna	pot**=4

Operadores relacionales

- Comparan datos. El resultado es True o False

Operador	Significado	Ejemplo
==	Comprueba si los operandos son iguales	print(3==4) #False
<	Operador menor	print(3<4) #True
<=	Operador menor o igual	print(3<=4) #True
>	Operador mayor	print(3>4) #False
>=	Operador mayor o igual	print(3>=1) #True
!=	Distinto	print(5!=5) #False

Operadores lógicos

- **Realizan operaciones lógicas AND, OR y NOT.**

Operador	Significado	Ejemplo
and	Si los dos operandos son True, el resultado será True	a=5;b=7 print((a<b)and(b==0)) #False
or	El resultado es True si alguno de los operandos lo es	a=5;b=7 print((a<b)or(b==0)) #True
not	Operador negado	a=5;b=7 print(not (a<b)) #False

Otros

- **Realizan operaciones varias con los datos en un programa. Se analizan en detalle al estudiar dichos puntos**

Operador	Significado
in	Comprueba si un dato está en una lista
is	En variables objeto, comprueba si ambas variables apuntan al mismo objeto

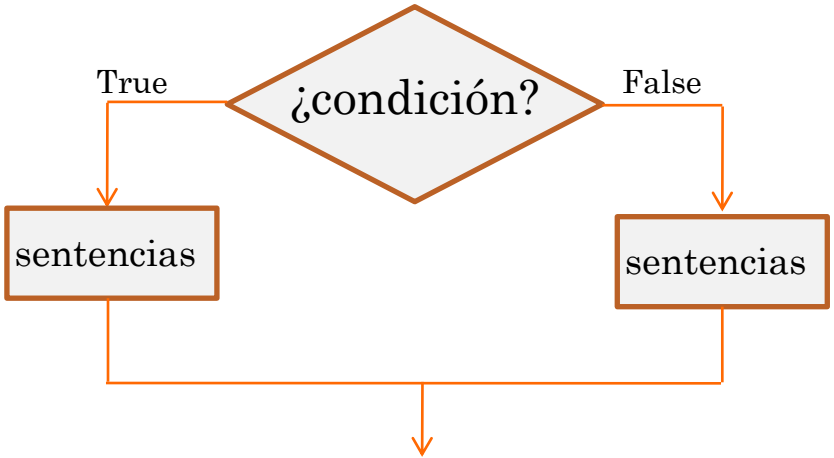
- **Ternario:**

```
data="positivo" if a>0 else"negativo"
```

Instrucción alternativa

Fundamentos

- **En función del resultado de una condición, que es interpretada como True o False, ejecuta uno u otro bloque de sentencias.**



Sintáxis

- **La instrucción condicional en Python es if else, y tiene el siguiente formato:**

```
if condicion:
        instruciones
else:
        instrucciones
```

- **El resultado de la condición se evalúa como verdadero si es True, distinto de 0 o no nulo**

- **Las instrucciones de cada bloque tienen que escribirse tabuladas**

- **El bloque *else* es opcional**

Ejemplos

```
n=5;a=7
if n>a:
        print ("El mayor es ",n)
else:
        print ("El mayor es ",a)
```

Muestra el mayor de dos números

```
a=input("nombre")
if a:
        print ("Te llamas "+a)
else:
        print ("sin nombre")
```

Si no introduce nada, se evalúa como falso

if elif

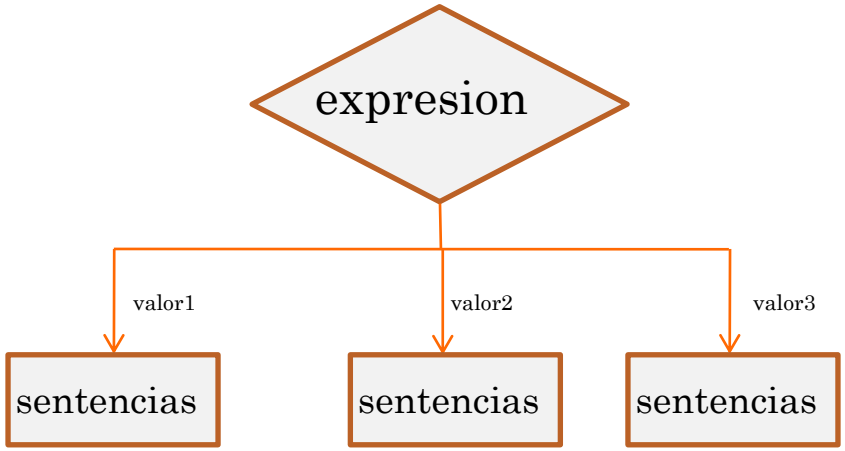
- **Cuando hay que preguntar por varias condiciones excluyentes entre sí, se puede utilizar la construcción if elif**

```
if condicion1:
        instruciones
elif condicion2:
        instrucciones
elif condicion3:
        instrucciones
:
```



Alternativa múltiple

- **Evalúa una expresión y en función de los poisbles resultados de esta puede ofercer distintos caminos**



Alternativa múltiple

- **Se materializa mediante la instrucción match:**

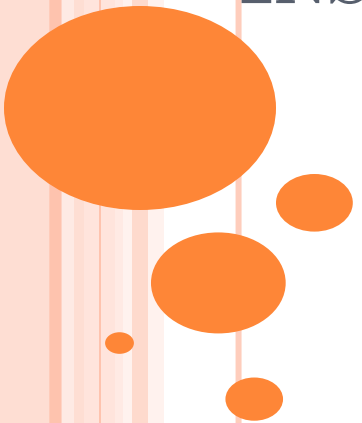
```
day = 4
match day:
  case 6:
    print("Hoy es sábado")
  case 7:
    print("Hoy es domingo")
  case _:
    print("Hoy no es fin de semana")
```

```
day = 4
match day:
  case 1 | 2 | 3 | 4 | 5:
    print("Hoy es día de diario")
  case 6 | 7:
    print("Es fin de semana")
```

- **Combinada con if:**

```
month = 5
day = 4
match day:
  case 1 | 2 | 3 | 4 | 5 if month == 4:
    print(" Hoy es día de diario  de abril")
  case 1 | 2 | 3 | 4 | 5 if month == 5:
    print(" Hoy es día de diario  de mayo")
  case _:
    print("Ninguno")
```

Instrucciones repetitivas



Características

- Ejecutan varias veces un grupo de sentencias
- Dos tipos:

for

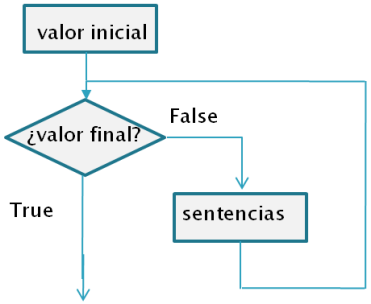
while

Instrucción for

- **Ejecuta un bloque de sentencias un número determinado de veces**

- **Apropiada para recorrer conjuntos de datos, como listas o tuplas:**

for variable in conjunto:
    instruciones



recorrido de un conjunto de datos

```
col=[4,8,20,56,1]
for n in col:
        print (n)
```

ejecuta las sentencias un número determinado de veces

```
for c in range(1,10):
    print("hello")
```

Establece un rango de valores numéricos, desde el primero hasta el anterior al último

Ejemplos for

```
suma=0
for n in range(1,10):
        suma=suma+n
print(suma)
```

suma los números comprendidos entre 1 y 9

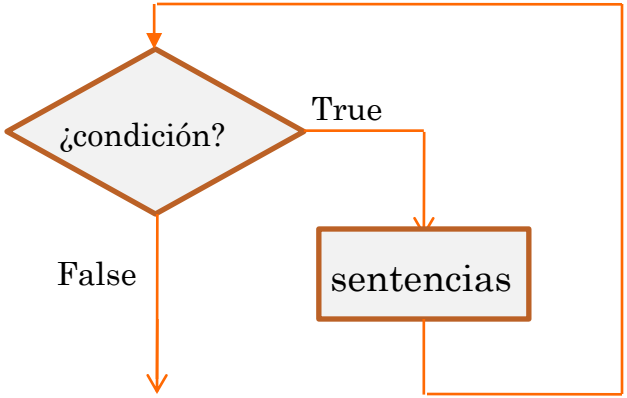
```
lista=[2,4,5,1,9,7]
for n in lista:
        if n%2==0:
                print (n, " es par")
        else:
                print (n, " es impar")
```

Recorre una lista y para cada número nos dice si es par o impar

Instrucción while

- **Ejecuta un bloque de sentencias mientras se cumple una condición**

```
while condicion:
        instrucciones
```



Ejemplo: solicita dato mientras sea negativo

```
edad=int(input("introduce edad"))
while edad<0:
        print("Valor incorrecto")
        edad=int(input("introduce edad"))

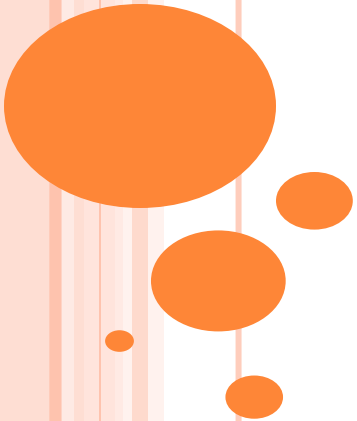
print("Su edad es ",edad," años")
```

Salida de un bucle

- **Existen dos sentencias que provocan la salida forzada de una instrucción repetitiva en Python:**
  - **break. Provoca la salida definitiva del bucle, pasando a ejecutar la siguiente instrucción del código**
  - **continue. Abandona la iteración actual, volviendo al principio del bucle. En el caso de while, evalúa de nuevo la condición**



Utilización de listas



¿Qué es una lista?

- **Se trata de un conjunto de datos separados por comas y encerrados entre corchetes:**

```
lista=[3,9,1,7,11,9]
```

- **Las listas son dinámicas, lo que significa que se pueden añadir y eliminar elementos de la misma en tiempo de ejecución.**

- **Pueden contener elementos de un mismo tipo o diferente.**
- **Se puede desempaquetar su contenido en variables individuales:**

```
lista=[3,9,5]
a,b,c=lista
```

Acceso a los elementos

- **Cada elemento de la lista tiene asignada una posición, siendo 0 la del primero:**

```
          0  1  2  3  4  5
lista=[3,9,1,7,11,9]
```

- **El acceso a cada dato se realiza con el nombre de la variable indicando la posición entre los corchetes:**

```
print (lista[3]) #7
lista[5]=15 #modifica la lista
```

- **También se puede obtener una sublista:**

```
print (lista[1:4]) # muestra [9, 1, 7]
```



Recorrido y consulta

- **Una lista puede ser recorrida con una instrucción for:**

```
for n in lista:
        print(n)
```

- **Se puede comprobar si un elemento se encuentra dentro de una lista utilizando el operador *in*:**

```
data=7
if data in lista:
        print ("Existe")
else:
        print ("No existe")
```



Funciones para listas

➢**Python dispone de una serie de funciones para trabajar con listas. Entre las más importantes:**

- **len. Calcula la longitud de una lista:**

```
print(len(lista)) #muestra 6
```

- **del. Elimina un elemento de la lista**

```
del (lista[0]) #borra el primero
```

- **max, min. Obtienen, respectivamente, el mayor y menor elemento de la lista:**

```
datos=[6,11,3,20,4,7]
print (max(datos)) # muestra 20
print (min(datos)) # muestra 3
```



Añadir elementos en una lista

- **Para añadir nuevos elementos a una lista se emplea el método *append()*.**

- **Ejemplo:**

```
datos=[6,11]
datos.append(5)
datos.append(7)
print (datos) # [6,11,5,7]
```

- **Mediante el método *insert()*, podemos añadir el elemento en una determinada posición:**

```
datos.insert(3,4)
print (datos) # [6,11,5,4,7]
```

- **También mediante los corchetes:**

```
lst=[3,2,8,11]
lst[0:1]=[2,9]
print(lst) #[2,9,2,8,11]
```



Compresión de listas

- **Permite generar una lista a partir de otra aplicando una operación**

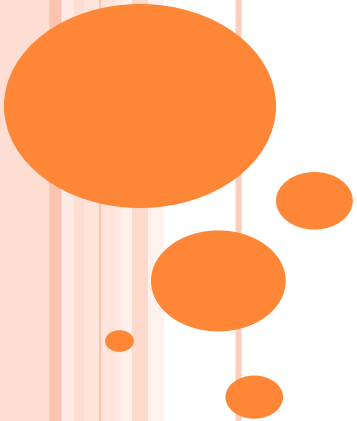
- **Ejemplo:**

```
datos=[3,8,1,2]
cuadrados=[n**2 for n in datos]
print(cuadrados)  #[9,64,1,4]

numeros = [1, 2, 3, 4, 5]
pares_cuadrados = [n**2 for n in numeros if n % 2 == 0]
print(pares_cuadrados)  #[4,16]
```



Utilización de tuplas



¿Qué es una tupla?

- **Es como una lista, solo que su contenido es fijo y no puede ser modificado.**
- **Se crean indicando los elementos entre paréntesis en lugar de corchetes**

```
mitupla=(4,9,3)
```

- **El acceso y recorrido se realiza igual que en una lista:**

```
mitupla=(4,9,3)
print(mitupla[0]) # 4
for n in mitupla:
        print(n)
mitupla[1]=20 #error
```



Convertir lista a tupla

- La función *tuple* permite convertir una lista en una tupla:

```
lista=[3,9,1,7,11]
mtup=tuple(lista)
```

- Las funciones len, max y min también son aplicables en tuplas

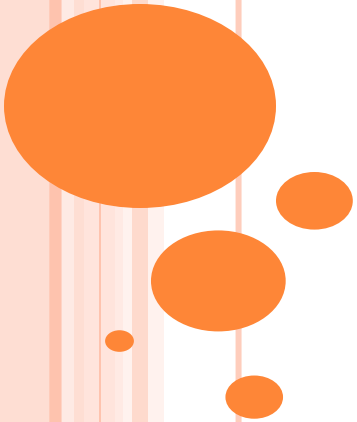
```
lista=[3,9,1,7,11]
mtup=tuple(lista)
print(len(mtup))  #5
print(max(mtup))  #11
print(min(mtup)) #1
```

- Con list transformamos una tupla en lista

```
tup=(3,9,1,7,11)
mlst=list(tup)
```



Diccionarios



¿Qué es un diccionario?

- **Agrupación de elementos formados por parejas clave:valor. Cada pareja se separa de la siguiente por una coma y todo el conjunto se encierra entre llaves :**

```
ciudades={"Madrid":27.3, "Sevilla":33.5, "Soria":19.0}
```

- **Las claves son únicas, por lo que no podrá haber dos elementos que tengan la misma clave.**



Acceso a los elementos

- **El acceso a los valores de un diccionario se realiza a través de la clave:**

```
print(ciudades["Madrid"]) #muestra 27.3
```

- **Los diccionarios son de lectura y escritura, por lo que se puede modificar el valor asociado a una clave:**

```
ciudades["Sevilla"]=31.7
```

- **Proporcionando una nueva clave, se añadiría la nueva pareja al diccionario:**

```
ciudades["Salamanca"]=22.4
```



Recorrido de un diccionario

- **Utilizando un *for* podemos recorrer las claves del diccionario:**

```
for n in ciudades.keys():
        print(n) #imprime los nombres de ciudades
```

- **También se pueden recorrer los valores:**

```
for n in ciudades.values():
        print(n) #imprime las temperaturas de ciudades
```

- **E incluso ambos a la vez:**

```
for k, v in ciudades.items():
    print(k,v) #imprime cada clave y su valor
```



Añadir y eliminar elementos

- **Para añadir nuevos elementos en un diccionario no tenemos más que acceder por una nueva clave y asignar un valor:**

```
ciudades["Toledo"]=30
```

- **Mediante la función *del()*, utilizada también en listas, podemos eliminar una entrada del diccionario:**

```
del (ciudades["Soria"]) #elimina la entrada cuya clave es Soria
```



Métodos de un diccionario

- **Los diccionarios proporcionan métodos o funciones internas para manipular estas estructuras de datos, entre ellos:**

  - **keys(), values(), items(). Comentados anteriormente, permiten recorrer un diccionario y acceder a su contenido**

  - **clear(). Elimina el contenido de un diccionario**

  - **update(otro_diccionario). Fusiona dos diccionarios:**

```
dic1={"x25":"Monitor", "as40":"Disco"}
dic2={"n10":"Impresora"}
dic1.update(dic2) #el diccionario dic1 tiene ahora tres elementos
```



Conjuntos

- **Grupo de elementos sin orden y sin posición, pero puede ser recorrido:**

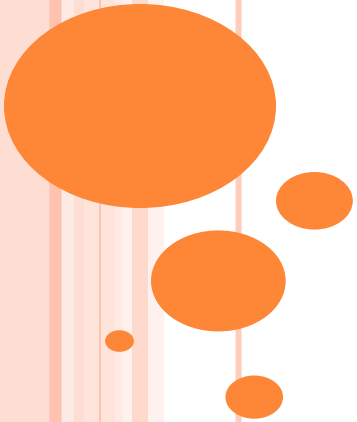
```
myset={"uno","dos","tres"}
for n in myset:
    print(n)   #los imprime todos pero en cualquier orden
```

- **No admiten duplicados:**

```
myset={"uno","tres","tres"}
for n in myset:
    print(n)   #imprime tres uno
print(len(myset)) #2
```

- **Dispone de los métodos *add()* y *remove()* para añadir nuevos elementos y eliminar existentes, respectivamente**

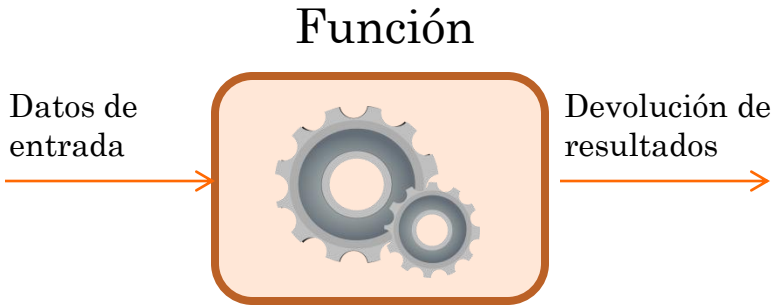
Funciones



Definición

- **Bloque de instrucciones predefinidas que realizan una determinada tarea**

- **Las funciones pueden recibir datos (parámetros) y devolver resultados**



Llamada a una función

- **Para ejecutar una función hay que llamarla por su nombre, indicando entre paréntesis los datos de entrada o parámetros:**

```
nombreFuncion(dato1, dato2)
```

- **Si la función devuelve un resultado se recoge en una variable en la misma instrucción de llamada:**

```
resultado=nombreFuncion(dato1, dato2)
```



Ejemplos de funciones Python

- **print e input son funciones:**

```
print("Esto es un mensaje")
```

Llamada a la función

El texto a mostrar es un parámetro de entrada

```
dato=input("Introduce nombre")
```

La función input devuelve un resultado que se guarda en una variable



Crear una función

- **Una función permite reutilizar código en distintas partes de un programa, además de organizar mejor el mismo**
- **Para crear una función propia en un programa python:**

```
def nombreFuncion(parametro1, parametro2,..):
    #instrucciones
```

- **Si la función va a devolver un resultado:**

```
def nombreFuncion(parametro1, parametro2,..):
    #instrucciones
    return resultado
```



Argumentos por defecto

- **Una función puede declarar parámetros con valores por defecto:**

```
def myfunction(x, y=100):
    print(x,":",y)
```

- **En caso de que no se envíe ese argumento en la llamada, tomará el valor por defecto:**

```
myfunction(30) #imprime 30:100
myfunction(20,45) #imprime 20:45
```



Número variable de argumentos

- **Se puede definir una función que reciba un número variable de parámetros. Estos se tratan como una tupla**

```
def myfunction(*args):
    for n in args:
        print(n)
```

- **Las siguientes serían llamadas válidas a myfunction:**

```
myfunction (2,6)
myfunction(8)
```

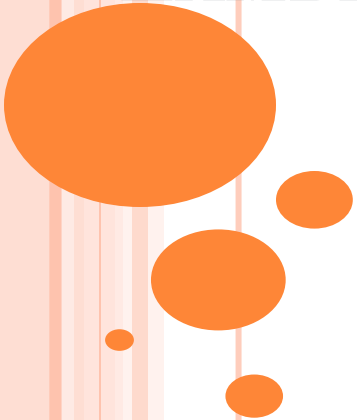
- **Se puede definir argumentos fijos y variables, éstos deben ser siempre el último:**

```
def f1(data,*args):

def f2(*args,n): #incorrecto
```



Lambdas y funciones especiales



Expresión lambda

- **Es una forma reducida de implementar una función**

```
def cuadrado(n):
    return n**2
```

como lambda ⇒

```
lambda n:n**2
```

- **La sintaxis es:**

argumentos de la función

código función sin return aunque devuelva resultado

```
lambda ar1, ar2:expresion
```

- **Se suelen utilizar como argumentos en llamadas a funciones que requieren como parámetro una función**



Funciones especiales

- **map(). Aplica una función a todos los elementos de una lista:**

```
numeros = [1, 2, 3, 4, 5]
cuadrados = list(map(lambda x: x**2, numeros))
print(cuadrados)  # [1, 4, 9, 16, 25]
```

- **filter(). Devuelve una sublista con los elementos que cumplen una condición**

```
numeros = [1, 2, 3, 4, 5]
pares = list(filter(lambda x: x % 2 == 0, numeros))
print(pares)  # [2, 4]
```

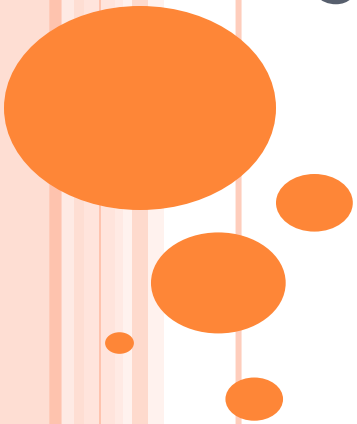
- **reduce(). Genera un resultado a partir de los elementos de una lista, combinándolos de dos en dos:**

```
from functools import reduce

numeros = [1, 2, 3, 4, 5]
suma = reduce(lambda a, b: a + b, numeros)
print(suma)  # 15
```



Cadenas de caracteres



El tipo cadena de caracteres

- **Una cadena de carácter es un texto, y se representa delimitando entre comillas los caracteres que la forman:**

```
texto="Esto es una cadena de caracteres"
```

- **El operador + puede utilizarse para unir (concatenar) cadenas de caracteres:**

```
t1="Hello"
t2=" Bye"
print(t1+t2)  #imprime Hello Bye
```

- **El operador * se utiliza para repetir varias veces una cadena:**

```
print(t1*3)  #imprime HelloHelloHello
```



Acceso a caracteres

- **Un texto puede tratarse como una tupla de caracteres:**

```
cad ="Mi texto"
print(cad[3]) #muestra 't'
print(cad[:5]) #muestra 'Mi te'
```

- **Al ser una tupla, no permite modificar su contenido:**

```
cad[6]="k" #error
```

- **Una cadena puede ser recorrida con un for:**

```
for car in cad:
        print(car) #imprime cada carácter
```



Métodos para manipular textos I

- **Los textos son objetos, por lo que exponen una serie de métodos para manipular dichos textos:**

  - **count(). Indica cuantas veces aparece una cadena dentro de otra. Además de la cadena a buscar, se indica el rango de posiciones de búsqueda:**

```
cad="Esto es una cadena de estas"
print(cad.count("a",0,13)) # 1
```

  - **endsWith(), startsWith. Indica si la cadena termina o empieza con la subcadena que se le pasa como parámetro. Se puede incluir el rango de posiciones de búsqueda**:

```
cad="Esto es una cadena de estas"
muestra="na"
print(cad.endswith(muestra))  #false
print(cad.endswith(muestra,0,11))  #true
```

Métodos para manipular textos II

- **find(). Busca una subcadena dentro de otra, devolviendo su posición. Si no la encuentra devuelve -1. Se puede incluir el rango de posiciones de búsqueda:**

```
cad="Esto es una cadena de estas"
print(cad.find("una")) #8
print(cad.find("una",10)) #-1, pues comienza a partir de la pos 10
```

- **isalpha(). Deuelve True si todos los caracteres de la cadena son alfabéticos, False si alguno no lo es**

- **isdigit(). Devuelve True si todos los caracteres de la cadena son numéricos**

- **isalnum(). Devuelve True si todos los caracteres son alfanuméricos**

- **len(texto). Se utiliza como una función, devuelve la longitud de la cadena que se le pasa como parámetro:**

```
cad="Esto es una cadena de estas"
cad="mi cadena"
print(len(cad)) #9
```

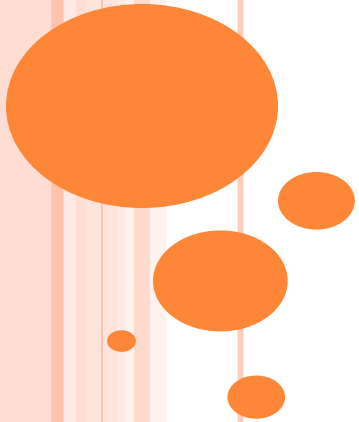
Métodos para manipular textos II

- **split(). Convierte una cadena en una lista formada por una serie de items, utilizando como separador la cadena que se proporciona como parámetro (por defecto es el espacio):**

```
dias="lunes,martes, miercoles, jueves"
print(dias.split(",")) #['lunes', 'martes', ' miercoles', ' jueves']
```



Fechas



El módulo datetime

- **El módulo datetime de python incluye una serie de objetos para trabajar con fechas y horas:**
  - **date. Objeto para trabajar con fechas**
  - **time. Objeto para trabajar con horas**
  - **datetime. Objeto para trabajar con fechas más horas**
  - **timedelta. Objeto que representa datos puntuales de una fecha-hora (días, meses, años, minutos, horas,..)**
- **Para poder utilizar el módulo es necesario importarlo:**

```
import datetime
```



Crear fechas

- **A través del método *today()* del objeto date podemos crear un objeto de fecha asociado al momento actual:**

```
import datetime
fechanow=datetime.date.today()
print(fechanow) #2020-11-10
```

- **Si queremos crear una fecha asociada a un momento dado:**

```
import datetime
otrafecha=datetime.date(1995,03,20)
print(otrafecha) #1995-03-20
```

- **Para crear un objeto que contenga fecha y hora actuales, utilizamos now() de datetime:**

```
import datetime
fechahoranow=datetime.datetime.now()
otrafechahora=datetime.datetime(2015,12,7,18,20)
print(fechahoranow) #2020-11-10 15:34:04.803111
print(otrafechahora) #2015-12-07 18:20:00
```



Manipular fechas

- **A través del objeto timedelta se puede sumar o restar una unidad de tiempo a una fecha:**

```
#suma 10 días a la fecha actual
f1=fechahoranow+datetime.timedelta(days=10)
print(f1) #2020-11-20 15:34:04.803111
```

- **Las fechas se pueden comparar utilizando los operadores python:**

```
print(f1>fechahoranow) #True
print(otrafechahora==fechahoranow) #False
```



Obtener elementos de una fecha

- **Mediante los atributos *year*, *month* y *day* del objeto date podemos obtener el años, mes y día asociado a la fecha, respectivamente:**

```
import datetime as dt
f=dt.date.today()
print(f.day,"/",f.month,"/",f.year)  #10/11/2020
```

- **Además de los atributos anteriores, el objeto datetime dispone de los atributos *hour*, *minute* y *second* para obtener la hora, minutos y segundo asociados a la fecha-hora.**



Caracteres de formato

- **Existen una serie de caracteres de formato que pueden ser utilizados de cara a presentar una fecha en un determinado formato, o para transformar un texto en un objeto de fecha:**

Carácter	Significado
%d	Día del mes (01-31)
%H	Hora (00-23)
%m	Mes (01-12)
%M	Minuto (00-59)
%S	Segundo (00-59)
%Y	Año formato largo

Presentación y parseado de fechas

- **Mediante el método *strftime()* de datetime se puede aplicar un formato a una fecha, indicando el objeto fecha y cadena de formato:**

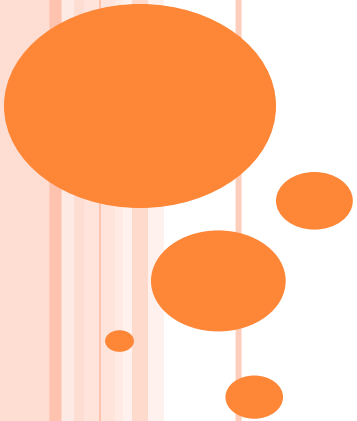
```
import datetime as dt
f=dt.date.today()
formato="%d/%m/%Y"
print(dt.datetime.strftime(f,formato)) #10/11/2020
```

- **Dada una cadena de caracteres, es posible transformarla a objeto fecha utilizando el método *strptime()* de datetime:**

```
import datetime as dt
formato="%d/%m/%Y"
texto="25/03/2003"
fecha=dt.datetime.strptime(texto,formato)
print(fecha) #2020-03-25 00:00:00
print(fecha.date()) # solo fecha: 2020-03-25
```



Acceso a ficheros



Fundamentos

- Un fichero es un archivo del ordenador donde se puede almacenar información para ser utilizada con posterioridad
- Python proporciona la función *open()* para trabajar con ficheros.
- Desde una aplicación Python es posible escribir en un fichero y leer su contenido



La función open

- **Permite abrir un fichero para realizar operaciones sobre el mismo. Devuelve un objeto fichero para operar sobre él:**

```
f=open(ruta_fichero,modo)
```

- **El primer parámetro es la ruta del fichero en el disco, mientras que el segundo representa la operación a realizar:**
  - **"r". Abre el fichero para operaciones de lectura**
  - **"w". Abre el fichero para operaciones de escritura, sobrescribiendo el contenido del mismo**
  - **"a". Abre el fichero para escritura, añadiendo nuevo contenido al existente**
- **En caso de incluir caracteres especiales:**

```
f=open(ruta_fichero,modo, encoding="UTF-8")
```

Escritura en un fichero

- **Se debe abrir el fichero en modo 'a' o 'w', según se quiera mantener el contenido o sobrescribir.**
- **Si el fichero no existe, se creará.**
- **Una vez abierto, se utiliza el método *write()* sobre el objeto fichero para escribir:**

```
#Escribe en modo append
f=open("info.txt", "a")
f.write("lunes\n")
f.write("martes\n")
f.close() #cierra el fichero
```

carácter \n para salto de línea

- **El método *close()* cierra el fichero al finalizar la operación.**

Lectura de un fichero

- **Se debe abrir el fichero en modo 'r'.**
- **Si el fichero debe existir, si no, se producirá un error.**
- **Una vez abierto, se utilizan los métodos:**
  - **readLine(). Devuelve la linea actual, situando el puntero en la siguiente línea. Si no hay ninguna línea devolverá cadena vacía:**

```
#Imprime todas las líneas del fichero
f=open("datos.txt","r")
cad=f.readline()
while cad!="":
    print(cad)
    cad=f.readline()
```

  - **readLines(). Devuelve una lista con todas las líneas del fichero.**

Captura de errores

- **Un error es una situación anómala que se puede producir durante la ejecución de un programa, por ejemplo, intentar leer de un fichero que no existe.**

- **Se puede capturar el error y definir un tratamiento personalizado del mismo utilizando el bloque *try except:***

```
try:
  #instrucciones
except tipo_error:
  #tratamiento error
```

- **Si las instrucciones del bloque try no provacan error, el programa se ejecuta con normalidad. Si se produce un error del tipo indicado en *tipo_error*, el programa salta al bloque *except***

Captura de excepciones en ficheros

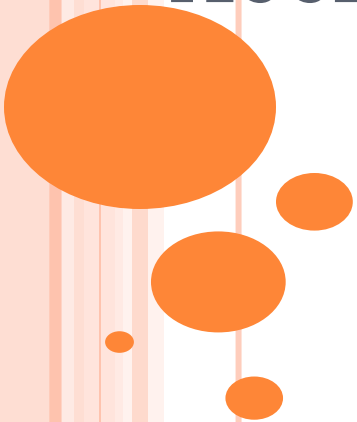
- **Si se intenta abrir un fichero como lectura y el fichero no existe, se producirá una excepción *FileNotFoundError***

- **Para tratar el error, utilizaremos el bloque *try except:***

```
try:
  r=open("fichero.txt", "r")
  #instrucciones para leer el contenido
  #del fichero
except FileNotFoundError:
  print("El fichero no existe")
```



Acceso a ficheros especiales



Formatos CSV y JSON

- Para almacenar datos en un fichero de texto que tengan una determinada estructura, se utilizan formatos especiales
- Entre los más utilizados están:
  - CSV
  - JSON



Ficheros CSV

- **csv(comma separate values) es un fichero de texto en el que cada entidad de datos (personas, empleados, libros,..) se almacena en una fila, separando cada dato del siguiente por carácter especial, habitualmente una coma:**

```
ana, ana@gmal.com, 2222
pedro, pedro@yahoo.com, 34234
....
```

- **En muchos casos, la primera línea puede contener los títulos:**

```
nombre, email, telefono
ana, ana@gmal.com, 2222
pedro, pedro@yahoo.com, 34234
....
```



Lectura de un fichero CSV

- **Python dispone del módulo csv para leer datos de un fichero que tenga este formato.**

- **La principal función de este módulo es *reader()*, que a partir del fichero devuelve un iterador sobre cada línea del fichero:**

```
import csv
f=open("personas.csv","r")
#cada elemento del reader es una lista
#con los datos de la fila
datos=csv.reader(f)
for p in datos:
    print(p)
```



```
['nombre', ' email', ' telefono']
['ana', ' ana@gmal.com', ' 2222']
['pedro', ' pedro@yahoo.com', ' 34234']
```

Función DictReader

- **Otra de las funciones del módulo csv , *DictReader()*, devuelve los resultados como diccionarios, en lugar de listas.**

- **La primera fila se emplea como claves del diccionario**

```
import csv
f=open("personas.csv","r")
dict=csv.DictReader(f)
for p in dict:
    print(p["nombre"])
```



```
ana
pedro
```

Formato JSON

- **JavaScript Object Notation (JSON), es una notación para la representación de objetos formados por parejas clave:valor.**

- **Originariamente empleada en JavaScript, es soportada por todos los lenguajes y tecnologías de desarrollo software**

- **En Python, un objeto JSON es un diccionario:**

```
{"nombre":"Juan", "edad":30,"email":"jan@gmail.com"}
```

- **JSON también se utiliza para representar arrays, que en python son listas:**

```
[4,9,13,20,5]
[{"nombre":"Juan", "edad":30,"email":"jan@gmail.com"},
{"nombre":"Maria", "edad":40,"email":"mar@gmail.com"},…]
```



Lectura de un fichero JSON

- **Python dispone del módulo json para leer datos de un fichero que tenga este formato y tratarlo directamente como diccionario/lista**

- **La principal función de este módulo es *load*, que a partir del fichero devuelve una lista o diccionario con los datos del documento:**

personas.json

```
[{"nombre":"Juan", "edad":30,"email":"jan@gmail.com"},
{"nombre":"Maria", "edad":40,"email":"mar@gmail.com"},…]
```

```
import json
f=open("personas.json","r")
#devuelve una lista de diccionarios
datos=json.load(f)
for p in datos:
    print(p["nombre"],"-",p["edad"])
```



```
Juan - 30
Maria - 40
```

Transformación de texto a JSON

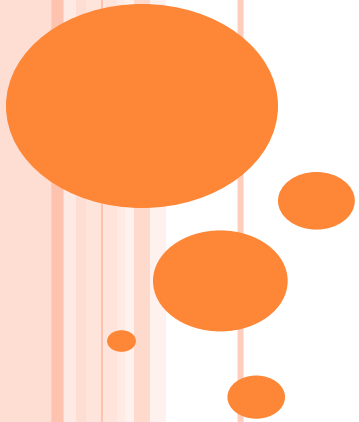
- **Dada una cadena de caracteres con formato JSON, el método *loads* del módulo json permite cargar en memoria un documento a partir de dicha cadena.**

```
import json
cad='[{"nombre":"Juan", "edad":30,"email":"jan@gmail.com"},{"nombre":"Maria",
"edad":40,"email":"mar@gmail.com"}]'
#devuelve una lista de diccionarios
datos=json.loads(cad)
for p in datos:
    print(p["nombre"],"-",p["edad"])
```

- **El documento JSON puede llevarse posteriormente a un fichero utilizando el método *dump()* de json:**

```
:
f=open("fichero.json","w")
json.dump(datos,f)
```

Pandas



¿Qué es pandas?

- **Módulo de python para manipular conjuntos de datos**
- **La estructura principal es el DataFrame: Representación de datos organizados de forma tabular, en filas y columnas**
- **El objeto DataFrame proporciona un amplio conjunto de métodos para operar con los datos (operaciones aritméticas, filtrados, ordenaciones, etc.)**
- **Para utilizar pandas debe ser importado:**

```
import pandas
```

- **O más comúnmente:**

```
import pandas as pd
```



Instalación

- **En caso de no disponer de este módulo, debemos instalarlo desde la línea de comandos con:**

```
>pip install pandas
```

- **O desde el terminal del propio IDE con**

```
!pip install pandas
```

- **En caso de no disponer de la utilidad pip, se puede descargar e instalar siguiendo este tutorial:**

https://tecnonucleous.com/2018/01/28/como-instalar-pip-para-python-en-windows-mac-y-linux/



Creación de un DataFrame I

- **Se puede crear un DataFrame desde distintas fuentes de datos:**
  - **Desde una lista de diccionarios. Cada diccionario de la lista contiene los datos de una fila, siendo las claves los nombres de las columnas:**

```
import pandas as pd
datos=[{"nombre":"paco","edad":20},
    {"nombre":"ana","edad":45},
    {"nombre":"maria","edad":30},
    {"nombre":"laura","edad":32}]
df=pd.DataFrame(datos)
print(df)
```

⇨

cada fila tiene un índice numérico

	edad	nombre
0	20	paco
1	45	ana
2	30	maria
3	32	laura

Si queremos que el índice de fila sea uno de los campos en lugar de un valor numérico:

```
df=pd.DataFrame(datos)
df.index=df["nombre"]
print(df)
```

⇨

nombre	edad	nombre
paco	20	paco
ana	45	ana
maria	30	maria
laura	32	laura

Creación de un DataFrame II

- **Desde un diccionario de listas. Las claves son los nombres de las columnas y los valores los datos de cada columna:**

```
import pandas as pd
datos={"ciudad":["Málaga","Ávila",
                 "Segovia","Alicante"],
       "temperatura":[24,17,18,27]}
df=pd.DataFrame(datos)
print(df)
```

⇨

	ciudad	temperatura
0	Málaga	24
1	Ávila	17
2	Segovia	18
3	Alicante	27

Creación de un DataFrame III

- **Desde un fichero CSV. Se utiliza la función read_csv() de pandas. La primera fila del fichero será interpretada como la que contiene los nombres de columnas:**

```
import pandas as pd
df=pd.read_csv("personas.csv", encoding="UTF-8")
print(df)
```

⇒

	nombre	email	telefono
0	ana	ana@gmal.com	2222
1	pedro	pedro@yahoo.com	34234

- **Desde un fichero JSON. Se emplea la función read_json() de pandas. Se interpreta como una lista de diccionarios:**

```
import pandas as pd
df=pd.read_json("contagios.json",encoding="UTF-8")
print(df)
```

⇒

	contagios	fecha	pais
0	14300	2020-10-30	España
1	29400	2020-11-07	Francia
2	65000	2020-10-30	EEUU

Información sobre DataFrame

- El objeto DataFrame proporciona una serie de atributos para obtener información sobre el mismo.

- En el ejemplo se muestran algunos de esos atributos:

```
import pandas as pd
datos=[{"nombre":"paco","edad":20}, {"nombre":"ana","edad":45},
    {"nombre":"maria","edad":30},{"nombre":"laura","edad":32}]
df=pd.DataFrame(datos)
print("columns: ",df.columns) #lista de nombres de columnas
print("values: ",df.values) #lista de valores [["paco",20],["ana",45]…]
print("head: ",df.head(2)) #muestra primeros datos
print("tail: ",df.tail(2)) #muestra últimos datos
```

Acceso al contenido de DataFram

- **El atributo iloc del DataFrame proporciona acceso a su contenido. A través de un índice se accede a cada fila:**

```
import pandas as pd
datos={"ciudad":["Málaga","Ávila","Segovia","Alicante"],
       "temperatura":[24,17,18,27]}
df=pd.DataFrame(datos)
print(df)
```

⇨

	ciudad	temperatura
0	Málaga	24
1	Ávila	17
2	Segovia	18
3	Alicante	27

```
print(df.iloc[0])
```

⇨

```
ciudad        Málaga
temperatura       24
```

```
print(df.iloc[0]["ciudad"])
```

⇨ Málaga

Iteración de un DataFrame

- **Mediante el método *iterrows()* del DataFrame se puede interar sobre cada pareja numero_fila-serie.**

- **Cada elemento de la serie puede ser accedido mediante índice o nombre:**

	edad	nombre
0	20	paco
1	45	ana
2	30	maria
3	32	laura

```
for k, v in df.iterrows():
    print(k,":",v[0])
```

⇒

```
0 : 20
1 : 45
2 : 30
3 : 32
```

```
for k, v in df.iterrows():
    print(k,":",v["nombre"])
```

⇒

```
0 : paco
1 : ana
2 : maria
3 : laura
```

Cálculos estadísticos

➢ **A partir de una columna del DataFrame, podemos realizar una serie de cálculos aplicando los siguientes métodos:**

- **count(). total de elementos de la columna**
- **max(), min(). Valor máximo y mínimo de la columna, respectivamente**
- **sum(). Suma de todos los valores de la columna**
- **mean(). Valor medio de los valores de la columna**

```
import pandas as pd
datos=[{"nombre":"paco","edad":20},{"nombre":"ana","edad":45},
        {"nombre":"maria","edad":30},{"nombre":"laura","edad":32}]
df=pd.DataFrame(datos)
print("Total elementos: ",df["edad"].count()) #4
print("Suma total: ",df["edad"].sum()) #127
print("Edad máxima: ",df["edad"].max()) #45
print("Edad mínima: ",df["edad"].min()) #20
print("Edad media: ",df["edad"].mean()) #31.75
```

Filtrado de elementos

- **Mediante el método *query()* del DataFrame obtenemos el subconjunto de elementos que cumplen la condición:**

personas.json

```
[{"nombre":"Juan", "edad":30,"email":"jan@gmail.com"},
{"nombre":"Maria", "edad":40,"email":"mar@gmail.com"},
{"nombre":"Elena", "edad":35,"email":"ele@gmail.com"},
{"nombre":"Jose", "edad":38,"email":"jose@gmail.com"}
]
```

```
import pandas as pd
df=pd.read_json("personas.json",encoding="UTF-8")
filtro=df.query("edad>35")
print(filtro)
```



```
   edad          email      nombre
1    40   mar@gmail.com  Maria
3    38  jose@gmail.com   Jose
```

- **Si el valor de comparación está en una variable:**

```
filtro=df.query("edad>@variable")
```

Agrupamientos

- **Con el método *groupby()* de DataFrame podemos agrupar datos por algún valor común de columna y aplicar algún tipo de cálculo estadístico sobre el grupo**

alumnos.json

```
[{"alumno":"Juan", "curso":"Java"},
{"alumno":"Pedro", "curso":"Java"},
{"alumno":"Elena", "curso":"Python"},
{"alumno":"Juan", "curso":"Net"}]
```

- **El campo de agrupación será el índice del resultado (Series o DataFrame) de aplicar un método al grupo**

```
import pandas as pd
df=pd.read_json("alumnos.json",encoding="UTF-8")
grupos=df.groupby("alumno")
#serie con el nombre de alumno como índice
#y total de cursos por alumno como columna
print(grupos["curso"].count())
```



```
alumno
Elena    1
Juan     2
Pedro    1
```

Cálculos sobre un grupo

- **Los métodos de cálculo se pueden aplicar sobre todas las columnas de un grupo (DataFrame) o sobre una columna en concreto (Series)**

empresa.csv

```
empleado,edad,salario,departamento
Juan,35,1200,ventas
Marcos,39,1300,informática
Raquel,28,1100,ventas
María,45,1700,informática
```

DataFrame

```
import pandas as pd
df=pd.read_csv("empresa.csv",encoding="UTF-8")
grupo=df.groupby("departamento")
print(grupo.mean()) #se aplica a todas las columnas
```

departamento	edad	salario
informática	42.0	1500.0
ventas	31.5	1150.0

Series

```
print(grupo["edad"].mean()) #se aplica a solo a esa columna
```

departamento	
informática	42.0
ventas	31.5

DataFrame

```
print(grupo.max())
```

departamento	empleado	edad	salario
informática	María	45	1700
ventas	Raquel	35	1200

Acceso al contenido del grupo

- En el caso de un DataFrame, se puede recorrer con iterrows().
- En el caso de un Series, se puede convertir a diccionario:

alumnos.json

```
[{"alumno":"Juan", "curso":"Java"},
{"alumno":"Pedro", "curso":"Java"},
{"alumno":"Elena", "curso":"Python"},
{"alumno":"Juan", "curso":"Net"}]
```

```
import pandas as pd
df=pd.read_json("alumnos.json",encoding="UTF-8")
totales=df.groupby("alumno")["curso"].count()
#se convierte a diccionario para poder recorrerlo
#la clave es el índice y el valor la columna
for k,v in dict(totales).items():
    print("Cursos de ",k,"=",v)
```

⇒

```
Cursos de  Elena = 1
Cursos de  Juan = 2
Cursos de  Pedro = 1
```

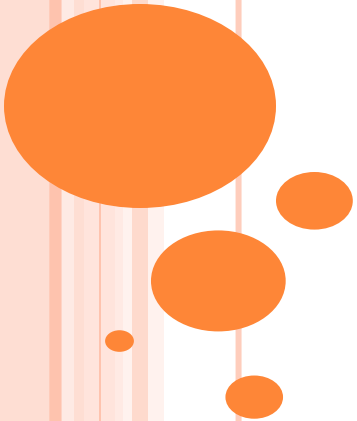
- En ambos casos, se puede usar el atributo *index* para acceder a los índices:

```
print(totales.sort_values(ascending=False).index[0])
```

⇒

```
Juan
```

Ordenación y transformación de datos



Ordenación DataFrame

- **El método *sort_values()* permite ordenar un DataFrame por un campo, indicando en el atributo *by* el campo de ordenación:**

empresa.csv

```
empleado,edad,salario,departamento
Juan,35,1200,ventas
Marcos,39,1300,informática
Raquel,28,1100,ventas
María,45,1700,informática
```

```
import pandas as pd
df=pd.read_csv("empresa.csv",
                encoding="UTF-8")
print(df.sort_values(by=["edad"]))
```



	empleado	edad	salario	departamento
2	Raquel	28	1100	ventas
0	Juan	35	1200	ventas
1	Marcos	39	1300	informática
3	María	45	1700	informática

```
import pandas as pd
df=pd.read_csv("empresa.csv",encoding="UTF-8")
print(df.groupby("departamento").mean().sort_values(by=["salario"]))
```



departamento	edad	salario
ventas	31.5	1150.0
informática	42.0	1500.0

- **El método se puede aplicar a un objeto *Series*, sin el *by*:**

```
print(df.groupby("departamento")["salario"].mean().sort_values())
```



departamento	
ventas	1150
informática	1500

Transformación de columnas

➢**Mediante el método *map()* de DataFrame es posible transformar los datos de una columna en otros, resultantes de la aplicación de una función:**

empresa.csv

```
empleado,edad,salario,departamento
Juan,35,1200,ventas
Marcos,39,1300,informática
Raquel,28,1100,ventas
María,45,1700,informática
```

Incrementa en un 10% el salario de todos los empleados

```
def multi(valor):
  return valor*1.10
df=pd.read_csv("empresa.csv",encoding="UTF-8")
df["salario"]=df["salario"].map(multi)
print(df)
```



	empleado	edad	salario	departamento
0	Juan	35	1320.0	ventas
1	Marcos	39	1430.0	informática
2	Raquel	28	1210.0	ventas
3	María	45	1870.0	informática

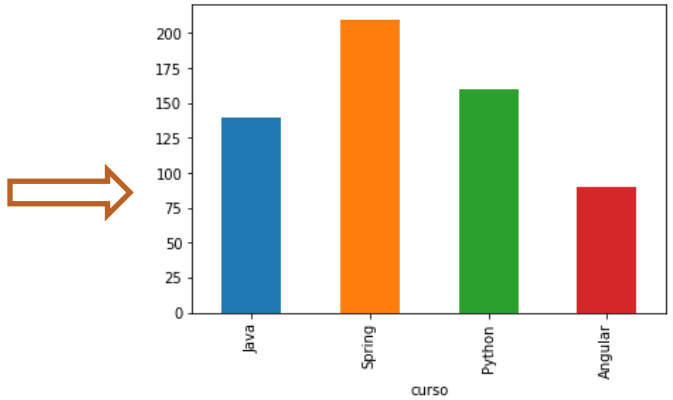
Graficos

- DataFrame dispone de un método *plot()* para representar el DataFrame o agrupamiento de DataFrame, en forma gráfica.
- A través del atributo *kind*, se establece el tipo de gráfico

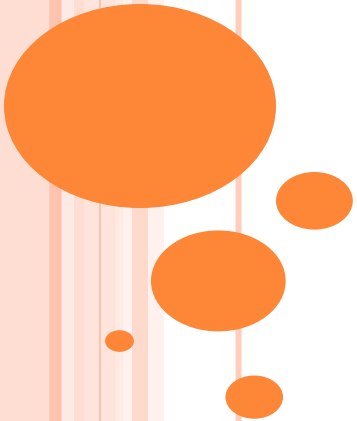
cursos.json

```
[{"curso":"Java","precio":140},
{"curso":"Spring","precio":210},
{"curso":"Python","precio":160},
{"curso":"Angular","precio":90}]
```

```
import pandas as pd
df=pd.read_json("cursos.json",encoding="UTF-8")
df.index=df["curso"]
print(df["precio"].plot(kind="bar"))
```



NUMPY



Introducción y uso

- **Módulo especializado en el cálculo numérico y análisis de datos, especialmente para un alto volumen de datos.**
- **A través de su objeto array podemos representar conjuntos de datos de un mismo tipo en varias dimensiones**
- **Se instala a través de PIP:**

```
>pip install NumPy
```

- **A partir de ahí ya podrá importarse como cualquier otro módulo:**

```
import numpy as np
```



Array NumPy

- **El objeto array, permite crear un array de n dimensiones para poder trabajar con estructuras de datos complejas.**
- **Para crear un array utilizaremos la función array() de NumPy:**

```
vector=np.array([2,8,10,5]  #array de una dimensión
tabla=np.array([[8,2],[5,6]]) #array de dos dimensiones
cubo=np.array(([[8,2],[5,5],[1,9]],[[-4,11],[18,1],[7,3]])) #array de tres dimensiones
```

- **El acceso a cada elemento del array se realiza indicando entre corchetes los índices:**

```
print(cubo[1,2,1]) #3
```

- **o también:**

```
print(cubo[1][2][1]) #3
```

- **Se puede acceder además a un rango de elementos:**

```
print(vector[1:3]) #[8,10]
```

- **Si queremos filtrar aquellos elementos que cumplen una condición, indicamos esta entre paréntesis:**

```
print(tabla[(tabla>5]) #[8,6]  Lo extrae como una array de una dimensión
```



Iterar un array NumPy

Para iterar un array utilizaremos un **bucle for** como en el caso de una lista.

```
m1=np.array([4,1,3,5,2])
m2=np.array([[8,2,7],[9,7,2],[2,9,5]])
for a in m1:
                print(a) # 4 1 3 5 2
for b in m2:
                for c in b:
                                print(c) #8 2 7 9 7 2 2 9 5
```

Para arrays grandes es más eficiente utilizar la función *nditer()*. También resulta más sencilla en el caso de arrays de varias dimensiones, ya que utiliza un único bucle for:

```
m1=np.array([4,1,3,5,2])
m2=np.array([[8,2,7],[9,7,2],[2,9,5]])
for a in np.nditer(m1):
                print(a) # 4 1 3 5 2
for b in np.diter(m2):
                print(b) #8 2 7 9 7 2 2 9 5
```

Si queremos convertir cada elemento del array en un tipo diferente, utilizamos el método *astype()* **:**

```
m1=np.array([4,1,3,5,2])
print(m1.astype(np.float64))  # [4. 1. 3. 5. 2.]
```

Operaciones aritméticas

El módulo numpy proporciona una serie de funciones para realizar operaciones aritméticas con arrays..

Estas funciones son:

- **add(a1,a2)**. Suma dos arrays (deben tener mismo tamaño y dimensión):

```
a1=np.array([3,5,7,10,1])
a2=np.array([1,0,4,8,2])
print(np.add(a1,a2)) #[4,511,18,3]
```

- **subtract(a1,a2)**. Resta al primer array el segundo:

```
a1=np.array([[3,5,7],[9,4,1]])
a2=np.array([1,2,5])
print(np.subtract(a1,a2))  # [[ 2  3  2]
                                  [ 8  2 -4]]
```

- **multiply(a1,a2)**. Multiplica el primer array por el segundo (cada subarray debe tener mismo tamaño):

```
a1=np.array([[3,5,7],[9,4,1]])
a2=np.array([1,2,5])
print(np.multiply(a1,a2))  # [[ 3 10 35]
                                  [ 9  8  5]]
```

- **divide(a1,a2)**. Divide el primer array entre el segundo

```
a1=np.array([[3,5,7],[9,4,1]])
a2=np.array([1,2,5])
print(np.divide(a1,a2))  # [[3.  2.5 1.4]
                    [9.  2.  0.2]]
```

Funciones estadísticas

El módulo numpy incluye algunas funciones con las que realizar útiles cálculos estadísticos sobre un array. A continuación comentamos las más interesantes:

- **amax(array) y amin(array)**. Devuelven, respectivamente, el mayor y el menor de los elementos de un array:

```
a1=np.array([[3,5,7],[9,4,1]])
print(np.amax(a1))  #9
print(np.amin(a1)) #1
```

- **average(array).** Calcula el valor medio a partir de todos los elementos del array:

```
a2=np.array([1,2,5])
print(np.average(a2))  # 2.6666
```

- **percentile(array,porcentaje)**. Calcula el percentil de un grupo de datos, que es el valor por debajo del cual caen un determinado porcentaje del grupo. Es un dato muy usado en estadística:

```
a2=np.array([1,7,5])
print(np.percentile(a2,30))  # 3.4
print(np.percentile(a2,80))  # 6.2
```



Búsquedas y ordenación

La realización de búsquedas y ordenaciones de datos en Python pueden realizarse de forma tremedamente eficiente a través de las funciones que proporciona la librería NumPy

- **sort(array)**. Devuelve el array ordenado. Si es de varias dimensiones, ordena cada dimensión independientemente:

```
a1=np.array([[3,2,7],[9,4,1]])
print(np.sort(a1)) #[[2 3 7] [1 4 9]]
#esto es una ordenación de mayor a menor
print(-np.sort(-a1)) #[[7 3 2]   [9 4 1]]
```

- **extract(condicion,array).** Extrae los elementos del array que cumplen la condición. Devuelve el resultado como un array de una dimensión:

```
a1=np.array([[3,2,7],[9,4,1]])
print(np.extract(a1>5,a1)) #[7,9]
```

- **unique(array)**. Devuelve un array de una dimensión con los elementos del array, eliminando duplicados:

```
a3=np.array([[8,2,7],[9,7,2],[2,9,5]])
print(np.unique(a3))   # [2 5 7 8 9]
```

